

Bathtub Racing Multiplayer

Final Report

Course: 4850 – Spring 2026

Instructor: Sharon Perry

Team Members:

William Pitts

Joshua Young

Thomas Powell

William Urvan

Date: May 2, 2026

Website URL: <https://multiplayertubracinggame.com/>

GitHub URL: <https://github.com/SpongyFall/Bathtub-Racing-Game>

STATS AND STATUS

Lines of Code	3886
Components/Tools	Unity, GoNet, Steam SDK, GitHub
Hours Estimated	164
Hours Actual	156
Status	Project 100% complete

1. Introduction	3
2. Challenges and Solutions	3
3. Requirements	4
Functional Requirements:.....	4
Non-Functional Requirements:.....	5
Constraints:.....	5
Environment:	5
Assumptions:	5
4. Analysis and Design	6
Design Considerations.....	6
Architectural Strategies	7
System Architecture	7
Detailed System Design	10
5. Development	11
Development Documentation:	12
Database Connection:	12
Project Setup:	12
6. Test Plan	13
Scope of Testing:	13
Test Environment:	14
Test Report	16
7. Version Control.....	18
8. Conclusion.....	19
9. Appendix	20

1. Introduction

This project extends an existing Unity-based bathtub racing game by adding multiplayer functionality through GoNet networking and Steam integration. The original version of the game supported only single-player gameplay, whereas this project focuses on enabling players to connect, race, and interact with one another in real-time.

The system integrates GoNet for real-time synchronization of player movement and game state, while the Steam SDK provides authentication and session management. A large portion of development effort was dedicated to ensuring the game could be launched and operated through the Steam platform, including appropriate session management and player activity tracking.

By adding a fully functional multiplayer experience, this project improves the game and builds on earlier work from the previous team.

2. Challenges and Solutions

One of the primary challenges in this project was integrating multiplayer functionality into an existing single-player codebase. The legacy structure of the original game required careful refactoring to support online gameplay. Changes made in one part of the system often affected other components, requiring a thorough understanding of the codebase before implementing new features. To address this, the team reviewed and analyzed the codebase before making modifications to ensure that new multiplayer logic could be integrated without breaking existing functionality.

Another major challenge was implementing real-time synchronization between players. To ensure that all players see consistent movements and interactions required handling latency and network delays. This was resolved by using

GoNet's synchronization tools along with interpolation techniques to smooth player movement and reduce the visual impact of latency.

The project also involved working with an earlier version of GoNet, which required additional effort to understand and integrate due to limited documentation and evolving features. This challenge was addressed through iterative testing and refinement, as well as guidance from the project sponsor.

3. Requirements

The following requirements are based on the Software Requirements Specification (SRS) developed for the project. These requirements define the expected functionality, performance, and constraints of the multiplayer bathtub racing system.

Functional Requirements:

- Launch:
 - The game must launch and display the main menu
 - Steam must display the user as actively playing the game
- Connectivity:
 - The system must allow players to host and join multiplayer sessions
 - A game server may be deployed to manage active multiplayer sessions
- Synchronization:
 - Players must be able to see other players moving and interacting in real time
 - The system must synchronize player movement and gameplay state across all connected clients
- Completion:
 - The system must support a complete race lifecycle, including start, progression, and completion

- The game must display race results and ranked winners upon completion

Non-Functional Requirements:

- Performance
 - The game must run efficiently on an average machine
- Error Handling
 - The system must produce minimal runtime or log errors during normal gameplay
- Usability
 - The user interface should remain consistent with the existing single-player version and provide a clear experience for multiplayer functionality

Constraints:

- The system must be developed using the Unity game engine
- Multiplayer must be implemented using the GoNet networking framework
- The system must be deployed and operated through the Steam platform

Environment:

- The system is designed for Windows-based environments
- Steam is required for authentication, session management, and game launch

Assumptions:

- The existing singleplayer system is functional and can be extended to support multiplayer
- Additional features beyond the core scope may be implemented if time allows

4. Analysis and Design

The system is built upon an existing single-player game developed by a previous capstone team and extends it to support multiplayer functionality. It relies on GoNet for networking and Steam services for authentication, lobby management, and session initialization. The system is designed to operate on Windows-based environments distributed through the Steam platform.

Design Considerations

The multiplayer integration builds upon an existing Unity-based game that provides core gameplay systems such as physics simulation, player controls, race progression, and user interface components. While these systems are largely functional, some components required modification to operate correctly in a multiplayer environment, as they were originally designed for single-player use.

The system depends on the GoNet networking framework for real-time multiplayer functionality and Steam services for authentication, lobby creation, and session management. The game is designed specifically for Windows 10 and 11 desktop environments and is distributed through the Steam platform. Users are expected to meet minimum hardware requirements to ensure acceptable performance during multiplayer sessions.

Several constraints influenced the design. The system must be developed using Unity, with multiplayer functionality implemented through GoNet and integrated with Steam. The networking model must follow GoNet's architecture, and the game must be launched through Steam to support authentication and multiplayer session creation. Development scope is also constrained by MVP requirements, prioritizing core multiplayer functionality and system stability over additional features.

Development followed an iterative approach, gradually adapting the existing single-player codebase to support multiplayer functionality. Core gameplay

systems were first adjusted to operate in a networked environment before introducing additional multiplayer features.

Architectural Strategies

The networking strategy follows GoNet's model, using networked entities to represent players and synchronize gameplay data. This approach was selected because GoNet is a required technology and provides built-in support for synchronization, authority handling, and multiplayer communication. By relying on this model, the system ensures consistent behavior and performance while simplifying integration with the existing codebase. However, this decision limits flexibility and postpones exploration of alternative networking approaches.

Steam services are used as the platform layer for authentication and multiplayer session management. Players can create and join sessions using Steam lobby features, including invitations and session discovery. This approach provides a familiar interface and simplifies session management while integrating directly with the networking layer. However, it restricts the system to the Steam platform and limits compatibility with other platforms.

To manage synchronization and latency, the system focuses on transmitting only gameplay-critical data, including player movement and race lifecycle events. Each player maintains local control of their movement, while key state updates are replicated across clients. Interpolation techniques are used to smooth remote player movement and reduce visible latency. More advanced networking techniques, such as client-side prediction and server reconciliation, were considered but not implemented due to increased complexity and risk within the project scope.

System Architecture

The system is decomposed into distinct subsystems to support multiplayer integration while maintaining the original single-player gameplay foundation. Responsibilities are divided across gameplay logic, networking communication,

server management, and session control to improve maintainability and reduce system dependency.

At a high level, the architecture consists of four primary layers: the gameplay layer, networking layer, dedicated server layer, and session control layer. These layers work together to manage player interaction, communication, and session flow.

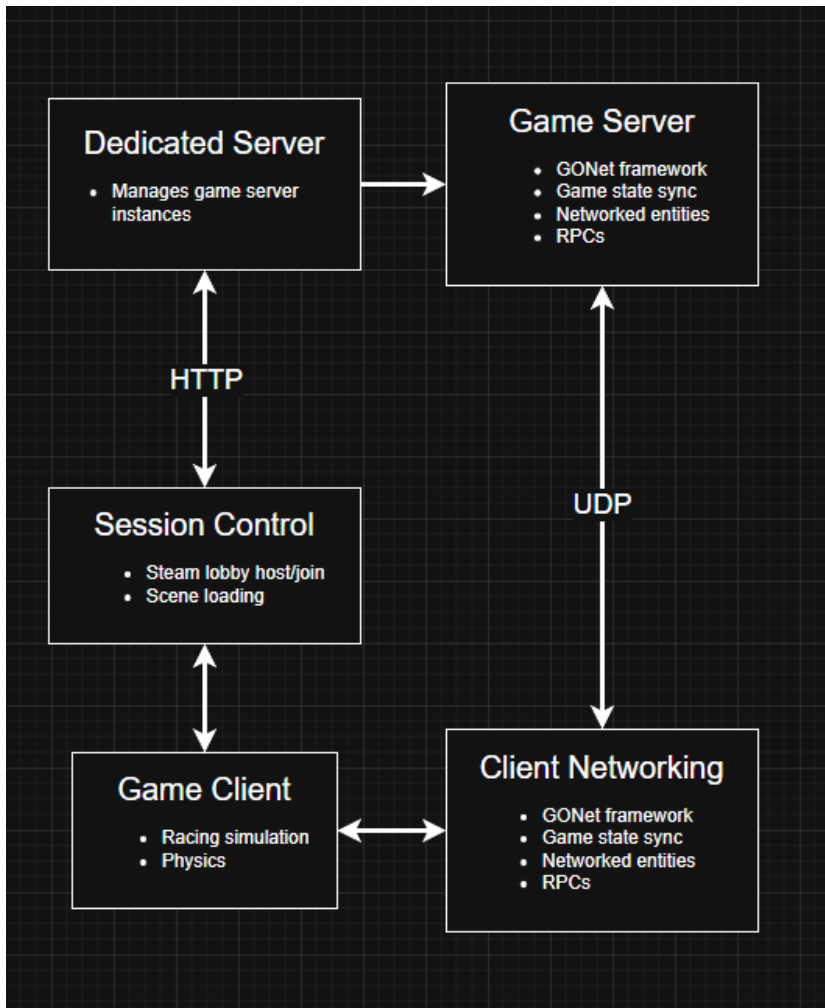


Figure 1: Multiplayer System Architecture Diagram

The Gameplay Subsystem is responsible for running the core racing simulation, including player movement, physics interactions, and race progression logic inherited from the single-player implementation.

The Networking Subsystem provides communication, state replication, and event messaging using the networking framework. It manages networked entities and ensures that gameplay remains consistent across all connected clients.

The Dedicated Server subsystem handles game server instances. When a player initiates a multiplayer session, a request is sent to start a server instance, which then returns connection details such as a port number. This information is distributed to all players, allowing them to connect to the same game session.

The Session Control subsystem manages the overall multiplayer lifecycle. It includes handling user identity through platform authentication, managing lobby states, and coordinating transitions between gameplay states. This subsystem ensures that all players move through the session lifecycle in a synchronized manner.

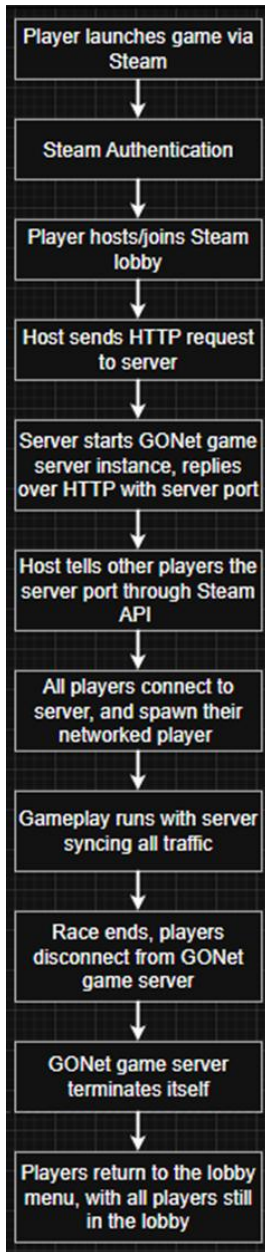


Figure 2: Multiplayer Session Lifecycle Flow Process

Detailed System Design

The system includes several key components that coordinate multiplayer functionality and gameplay behavior.

The Session Manager acts as a central controller for multiplayer sessions and persists across scene transitions. It manages the lifecycle of a session, including lobby state, race initialization, active gameplay, and cleanup. It ensures that all players are properly connected before a race begins and that transitions occur consistently across all clients.

The Steam Lobby Manager handles session discovery and player interaction within the platform. It allows players to create and join lobbies, manage invitations, and maintain the player roster. It ensures that lobby state remains consistent and integrates with the networking layer to initialize gameplay sessions.

The GoNet Network Controller is responsible for initializing and managing the networking session. It controls connection state, handles the spawning and synchronization of networked entities, and ensures that networking is properly started and terminated as sessions begin and end.

The Networked Player Controller adapts the existing player movement and physics systems for multiplayer use. It distinguishes between local and remote players, allowing local players to generate input while remote players are updated based on synchronized state. This ensures consistent gameplay while maintaining responsive controls.

The Race Manager controls the multiplayer race lifecycle. It manages race initialization, countdown timing, active race state, finish conditions, and result display. It ensures that all players begin and end races at the same time and that race results are applied consistently across all clients.

5. Development

The development process involved modifying the existing codebase to support multiplayer functionality. Core gameplay components were adapted to operate in

a networked environment which ensures consistent synchronization and interaction between players.

Development Documentation:

Multiplayer functionality was implemented using GoNet's networking framework. Networked player objects were created to represent each connected user, with synchronization of player movement and gameplay state handled through GoNet's entity system. The implementation distinguishes between local and remote players, allowing local players to provide input while remote players are updated based on synchronization data.

Steam integration was implemented to manage user authentication and multiplayer session flow. Players launch the game through Steam, which verifies user identity and enables access to multiplayer features. Steam lobby services are used to create and join sessions prior to entering gameplay.

Additional development effort was required to ensure proper integration with the Steam platform. This included configuring the Steam environment, verifying that the game launches correctly through Steam, and ensuring that session creation and joining function reliably.

Database Connection:

Not applicable, no database used for project.

Project only utilizes GoNet and Steam services

Project Setup:

To set up and run the project:

1. Install Unity (version 2023.2.22f1)
2. Clone the project repository
3. Open the project using Unity Hub
4. Configure GoNet & Steam

5. Launch project through Unity Hub

6. Test Plan

The purpose of this test plan is to verify that the multiplayer system functions correctly and reliably across its core features. Testing focuses on validating session creation, player synchronization, race mechanics, and overall system stability.

Scope of Testing:

In Scope

- Multiplayer sessions setup, including creating and joining a race
- Core gameplay loop from race start to start finish
- Player movement, steering, and general control behavior
- Synchronization of player positions and basic game state across clients
- Race mechanics such as lap tracking and finish conditions
- Collision handling with track boundaries and obstacles
- Basic handling of player disconnects during a session
- Integration points required to start and run a multiplayer session
- General performance, responsiveness, and stability during normal gameplay

Out of Scope

- Additional tracks, modes, or extended content beyond base implementation
- Large-scale stress testing or long-term load/performance analysis

Test Environment:

Hardware	PC used for testing
Operating System	Windows 11
Game Build	[Build number or date]
Input Devices	Keyboard and Mouse
Network	Internet connection for multiplayer testing
Unity Version	2023.2.22f1

Test Case ID	Description	Expected Result
TC-01	Launch game and open main menu	Main menu appears without errors and responds to input.
TC-02	Create a multiplayer session	New race session is created without errors
TC-03	Join existing multiplayer session	Player successfully joins the selected session

TC-04	Start a race with multiple players connected	Race begins correctly for all connected players
TC-05	Verify synchronization of player positions	Player positions update consistently across connected clients
TC-06	Collide with track boundary or obstacle	Collision is detected and handled as intended without breaking gameplay
TC-07	Verify lap tracking	Lap progress updates correctly when finish line is crossed
TC-08	Race completion	Race completion is detected and results are displayed correctly
TC-09	Return to main menu from gameplay	Game returns to the main menu without crashing or freezing
TC-10	Disconnect one player during a session	Remaining session handles the disconnect appropriately
TC-11	Verify general performance during normal gameplay	Game remains responsive and stable during standard gameplay
TC-12	Verify usability of menus and session flow	Menus and session setup are understandable and function without major confusion

Test Report

The software test report summarize the results of each executed test case and evaluates whether the system behaved as expected under normal gameplay conditions.

Test Case ID	Description	Expected Result	Pass/Fail
TC-01	Launch game and open main menu	Main menu appears without errors and responds to input.	Pass
TC-02	Create a multiplayer session	New race session is created without errors	Pass
TC-03	Join existing multiplayer session	Player successfully joins the selected session	Pass
TC-04	Start a race with multiple players connected	Race begins correctly for all connected players	Pass
TC-05	Verify synchronization of player positions	Player positions update consistently across connected clients	Pass
TC-06	Collide with track boundary or obstacle	Collision is detected and handled as intended without breaking gameplay	Pass
TC-07	Verify lap tracking	Lap progress updates correctly when finish line is crossed	Pass

TC-08	Race completion	Race completion is detected and results are displayed correctly	Pass
TC-09	Return to main menu from gameplay	Game returns to the main menu without crashing or freezing	Pass
TC-10	Disconnect one player during a session	Remaining session handles the disconnect appropriately	Pass
TC-11	Verify general performance during normal gameplay	Game remains responsive and stable during standard gameplay	Pass
TC-12	Verify usability of menus and session flow	Menus and session setup are understandable and function without major confusion	Pass

Overall, the test cases passed successfully, confirming that the core multiplayer functionality operates as intended. Initial testing showed that issues occurred with lap tracking and race completion. These issues were resolved after additional testing and code review.

7. Version Control

We chose GitHub for our version control. Because this project uses Unity, we added the appropriate .gitignore to avoid uploading heavy files such as the 'library' file generated by the game engine.

Version control grants the ability to sync files with ease, create branches for organization, provide a reversible history, and show each commented change. This is especially useful when collaborating with multiple developers on the same source code.

Our repository: <https://github.com/SpongyFall/Bathtub-Racing-Game>

Commit History

Here is a summarized history of all pushes to the repo:

- Updated project to Unity 2023.2.22f1 and integrated the GONet SDK.
- Implemented Steam lobby system with hosting, joining, and random matchmaking functionality.
- Added lobby chat, session handshake, and overall network state management using Steamworks and GONet.
- Developed multiplayer synchronization systems for player karts, AI, and race data across clients.
- Created kart spawning system based on Steam lobby player order with deterministic placement.
- Implemented race management system to coordinate player spawning, countdowns, and game flow.
- Fixed major multiplayer issues including spawning ownership, desyncs, and lobby state inconsistencies.
- Added support for joining via lobby codes and improved lobby UI and navigation.
- Implemented controller support, input system improvements, and UI navigation (out of scope).

- Redesigned AI system using spline-based path following for more reliable behavior (out of scope).
- Fixed numerous race logic bugs including lap counting, waypoint tracking, and player placement accuracy (out of scope).
- Improved race start synchronization by ensuring all players are spawned before countdown begins.
- Added host-side validation and handling for disconnected or unresponsive players.
- Implemented full game loop including track selection, lap configuration, AI count, and end game screen (out of scope).
- Added lobby join/leave notifications and improved cleanup on disconnect.
- Integrated and synchronized kart customization system with save/load functionality (out of scope).

8. Conclusion

The project successfully extends the existing singleplayer racing game to support multiplayer functionality. The system enables players to connect, interact, and compete in real time through integrated networking and session management. Core requirements were met which resulted in a stable and functional multiplayer experience, with some additional enhancements beyond the original scope also implemented.

Throughout development, challenges related to adapting singleplayer systems for multiplayer use, including synchronization and consistency across clients, were addressed through iterative development and testing. The finished system now provides a solid foundation for future work while also supporting the goal of increasing engagement between SPSU alumni and KSU.

9. Appendix

A. Out-of-Scope Additions

In addition to the core multiplayer functionality, several features were implemented:

1. More Advanced AI Opponents
 - a. Enhancements were made to AI behavior to improve gameplay experience and provide more competitive interactions during races.
2. Controller Connectivity
 - a. Support for external controllers was implemented, allowing players to use alternative input devices.
3. Tub Customization Fixes
 - a. Improvements were made to the tub customization system to correct issues and ensure consistent functionality across sessions.

B. Project Plan

Project Overview

This project is a continuation of an earlier industry capstone project in which students developed a bathtub racing game using the Unity game engine. The objective of this phase of the project was to extend the existing single-player system by implementing multiplayer functionality, including networking and Steam integration.

Project Repository

<https://github.com/SpongyFall/Bathtub-Racing-Game>

Meeting Schedule

Biweekly meetings were planned with the industry sponsor. Meeting times were to be determined based on sponsor availability.

Collaboration and Communication Plan

The team utilized multiple communication platforms to coordinate development and maintain consistent collaboration:

- Microsoft Teams for biweekly meetings with sponsors
- Text communication for coordination with the course instructor
- Discord for internal team communication
- Microsoft Office tools (Word, Excel, etc.) for shared documentation

Version Control Plan

The project was managed through a private GitHub repository in accordance with industry sponsor requirements. Version control was used to track changes, manage contributions, and maintain a consistent development history.

Project Deliverables

The project was structured into three major milestones:

Milestone 1 — Requirements and Multiplayer Architecture

- Requirements engineering documentation
- Functional and non-functional requirements
- Multiplayer constraints
- Architecture design and networking flow diagrams
- Codebase assessment and refactor planning
- Risk identification
- Initial repository setup and commits

Milestone 2 — Multiplayer Prototype Implementation

- Integration of networking framework
- Multiplayer race prototype

existing code before implementing new features or making significant changes.